

Lab 05: Monitoring and Observability with Prometheus and Grafana

Overview

You've containerized your ML application in Lab 04. But how do you know if it's working correctly in production? How do you know when something goes wrong before users complain? This is where **monitoring and observability** come in.

In this lab, you will add a complete monitoring stack to your ML application using industry-standard tools: **Prometheus** for metrics collection and **Grafana** for visualization.

Learning Objectives

After completing this lab, you will be able to:

1. Explain why monitoring is critical for production ML systems
2. Instrument a Flask application to expose Prometheus metrics
3. Configure Prometheus to collect metrics from your application
4. Create Grafana dashboards to visualize metrics
5. Write and trigger alerting rules
6. Use PromQL to query time-series data

Prerequisites

- Completed Labs 01-04 (working containerized Flask API)
 - Docker and docker-compose installed and running
 - Basic understanding of YAML configuration files
-

Part 1: Why Monitor ML Systems?

The Problem: Blind Deployments

Consider this scenario: You deploy your churn prediction API. It works fine in testing. A week later, your business team complains that predictions seem wrong. You check the logs and discover:

- The API has been returning errors for 3 days
- Memory usage spiked and caused slowdowns
- A model file got corrupted during a deployment

Without monitoring, you're flying blind. You only learn about problems when users complain - or worse, when the business impact is already done.

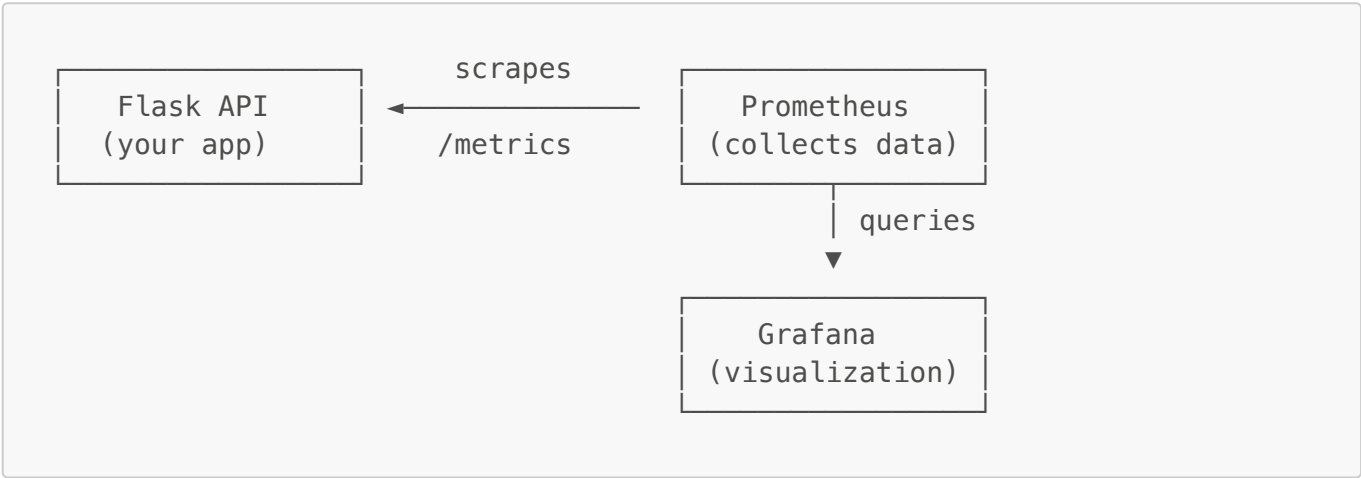
The Solution: Observability

Observability means having insight into your system's internal state through external outputs. For ML systems, we care about:

Category	What to Monitor	Why It Matters
Availability	Is the API responding?	Users can't get predictions if it's down
Performance	How fast are predictions?	Slow APIs frustrate users and timeout
Errors	What's failing and why?	Catch problems before they spread
Usage	How many predictions?	Capacity planning, cost estimation
Model Behavior	Prediction distribution	Detect model drift or data issues

The Monitoring Stack

We'll use two complementary tools:



- **Prometheus:** Collects and stores metrics as time-series data
- **Grafana:** Visualizes metrics through dashboards and alerts

Part 2: Understanding Prometheus Concepts

Before writing code, let's understand the core concepts.

How Prometheus Works

Prometheus uses a **pull-based model**:

1. Your application exposes metrics at an HTTP endpoint (**/metrics**)
2. Prometheus periodically "scrapes" this endpoint (e.g., every 15 seconds)
3. Prometheus stores the metrics as time-series data
4. You query the data using PromQL (Prometheus Query Language)

This is different from a push model where applications send metrics to a central server. The pull model is simpler - your app just needs to expose data; Prometheus handles collection.

Metric Types

Prometheus has four metric types. Understanding these is crucial:

1. Counter

A value that only goes **up** (or resets to zero on restart).

```
# Example: Total predictions made
ml_predictions_total{status="success"} 1523
ml_predictions_total{status="error"} 47
```

Use for: Request counts, error counts, tasks completed

2. Gauge

A value that can go **up or down**.

```
# Example: Current memory usage
app_memory_usage_bytes 256000000
```

Use for: Memory usage, CPU percentage, queue size, temperature

3. Histogram

Tracks the **distribution** of values in configurable buckets.

```
# Example: Request duration distribution
ml_prediction_duration_seconds_bucket{le="0.1"} 980 # 980 requests under
100ms
ml_prediction_duration_seconds_bucket{le="0.5"} 1495 # 1495 requests
under 500ms
ml_prediction_duration_seconds_bucket{le="1.0"} 1520 # 1520 requests
under 1s
```

Use for: Latency, request sizes - anything where you want percentiles

4. Summary

Similar to histogram but calculates quantiles on the client side. We'll focus on histograms in this lab.

Labels: Adding Dimensions

Labels let you slice metrics by different dimensions:

```
ml_predictions_total{model_version="v1", prediction_result="Yes",
status="success"} 523
ml_predictions_total{model_version="v1", prediction_result="No",
status="success"} 891
```

```
ml_predictions_total{model_version="v2", prediction_result="Yes",
status="success"} 234
```

With labels, one metric can answer many questions:

- Total predictions: `sum(ml_predictions_total)`
- Predictions by model: `sum(ml_predictions_total) by (model_version)`
- Error rate: `sum(ml_predictions_total{status="error"}) / sum(ml_predictions_total)`

Part 3: Instrumenting Your Flask Application

Now let's add metrics to your Flask API. We'll build this incrementally.

Step 1: Add Dependencies

Add these packages to your `requirements.txt`:

```
prometheus-client==0.21.0
prometheus-flask-exporter==0.23.1
psutil==6.1.0
```

- `prometheus-client`: Core Prometheus library for Python
- `prometheus-flask-exporter`: Flask integration that auto-creates a `/metrics` endpoint
- `psutil`: For monitoring system resources (CPU, memory)

Step 2: Basic Setup - Expose the /metrics Endpoint

Start with the simplest possible instrumentation. Add these imports and initialization to your `src/app.py`:

```
from prometheus_flask_exporter import PrometheusMetrics

app = Flask(__name__)

# This single line does a lot:
# 1. Creates a /metrics endpoint
# 2. Automatically tracks request count and latency for all endpoints
metrics = PrometheusMetrics(app)

# Add application metadata as a metric
metrics.info('app_info', 'ML API Information', version='1.0.0',
app_name='churn-prediction-api')
```

What this gives you for free:

- `flask_http_request_total` - count of all HTTP requests

- `flask_http_request_duration_seconds` - latency histogram
- `flask_http_request_exceptions_total` - exception counts

Test it: Start your app and visit `http://localhost:5000/metrics`. You'll see Prometheus-formatted metrics!

Step 3: Add Custom ML Metrics

The auto-generated metrics are useful, but we want ML-specific insights. Add custom metrics:

```
from prometheus_client import Counter, Histogram, Gauge

# Counter: Track total predictions with labels for segmentation
prediction_counter = Counter(
    'ml_predictions_total',          # Metric name
    'Total number of predictions made',  # Help text (description)
    ['model_version', 'prediction_result', 'status']  # Label names
)

# Histogram: Track prediction latency with custom buckets
prediction_latency = Histogram(
    'ml_prediction_duration_seconds',
    'Time spent processing prediction requests',
    ['model_version'],
    buckets=[0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1.0, 2.5, 5.0]  # Bucket
    boundaries in seconds
)

# Gauges: Track current system state
memory_usage_gauge = Gauge('app_memory_usage_bytes', 'Memory usage of the
application')
cpu_usage_gauge = Gauge('app_cpu_usage_percent', 'CPU usage percentage')
model_loaded_gauge = Gauge('model_loaded', 'Whether models are loaded',
    ['model_version'])
```

Step 4: Record Metrics in Your Prediction Function

Now instrument your `make_prediction` function to record these metrics.

Important: Record metrics for ALL outcomes - successes, validation errors, and exceptions. This was a common mistake: only recording successful predictions means you can't alert on errors!

```
def make_prediction(json_data, model, model_version):
    start_time = time.time()  # Start timing immediately

    # Handle missing input
    if json_data is None:
        # Record this error in metrics!
        prediction_counter.labels(
            model_version=model_version,
```

```

        prediction_result='no_input',
        status='error'
    ).inc()
    return {"error": "No input data provided"}, 400

# ... validation code ...

# For validation errors, also record metrics:
error_msg, status_code = validate_input(item)
if error_msg:
    prediction_counter.labels(
        model_version=model_version,
        prediction_result='validation_error',
        status='error'
    ).inc()
    return {"error": error_msg}, status_code

try:
    # ... prediction logic ...

    # Record successful predictions
    for result in results:
        prediction_counter.labels(
            model_version=model_version,
            prediction_result=result['prediction'],
            status='success'
        ).inc()

    # Record latency
    duration = time.time() - start_time

prediction_latency.labels(model_version=model_version).observe(duration)

    return results, 200

except Exception as e:
    # Record exceptions
    prediction_counter.labels(
        model_version=model_version,
        prediction_result='error',
        status='error'
    ).inc()
    raise

```

Step 5: Add System Resource Monitoring

Add a background thread to periodically update system metrics:

```

import psutil
import threading
import os

```

```
def monitor_system_resources():
    """Background thread to monitor system resources every 15 seconds."""
    while True:
        try:
            process = psutil.Process(os.getpid())
            memory_usage_gauge.set(process.memory_info().rss)
            cpu_usage_gauge.set(process.cpu_percent(interval=1))
        except Exception:
            pass # Don't let monitoring errors crash the app
        time.sleep(15)

# Start this thread when the app runs
if __name__ == '__main__':
    monitor_thread = threading.Thread(target=monitor_system_resources,
    daemon=True)
    monitor_thread.start()

    port = int(os.environ.get('PORT', 5000))
    app.run(host='0.0.0.0', port=port, debug=False)
```

Part 4: Setting Up Prometheus

Your Flask app now exposes metrics. Next, we need Prometheus to collect them.

Step 1: Create Directory Structure

```
mkdir -p prometheus/rules
```

Step 2: Create Prometheus Configuration

Create `prometheus/prometheus.yml`:

```
# Global configuration
global:
    scrape_interval: 15s # How often to collect metrics
    evaluation_interval: 15s # How often to evaluate alert rules

# Load alert rules from files
rule_files:
    - "/etc/prometheus/rules/*.yml"

# Define what to scrape
scrape_configs:
    # Prometheus monitors itself
    - job_name: 'prometheus'
      static_configs:
        - targets: ['localhost:9090']
```

```
# Your ML API
- job_name: 'ml-api'
  scrape_interval: 5s # More frequent for the API
  static_configs:
    - targets: ['app:5000'] # 'app' is the Docker service name
  metrics_path: /metrics
```

Key points:

- **scrape_interval**: How often Prometheus pulls metrics (balance freshness vs. load)
- **targets**: Use Docker service names, not **localhost** (containers have their own network)
- **metrics_path**: Defaults to **/metrics** but can be customized

Step 3: Create Alert Rules

Alerts let Prometheus notify you when conditions are met. Create **prometheus/rules/alerts.yml**:

```
groups:
  # Production alerts - real issues
  - name: ml_api_alerts
    rules:
      # Alert when API is completely down
      - alert: APIDown
        expr: up{job="ml-api"} == 0
        for: 30s
        labels:
          severity: critical
        annotations:
          summary: "ML API is down"
          description: "The ML API has been unreachable for over 30
seconds."

      # Alert on high error rate
      - alert: HighErrorRate
        expr: |
          (sum(rate(ml_predictions_total{status="error"}[2m]))
           / sum(rate(ml_predictions_total[2m]))) > 0.1
        for: 1m
        labels:
          severity: warning
        annotations:
          summary: "High prediction error rate"
          description: "Error rate exceeds 10% over the last 2 minutes."

      # Alert on slow predictions
      - alert: HighLatency
        expr: |
          histogram_quantile(0.95,
sum(rate(ml_prediction_duration_seconds_bucket[2m])) by (le)) > 0.5
        for: 1m
```



```
labels:
  severity: warning
annotations:
  summary: "High prediction latency"
  description: "95th percentile latency exceeds 500ms."

# Demo alerts - easy to trigger for testing
- name: lab_demo_alerts
  rules:
    # Fires when 3+ errors occur (easy to trigger)
    - alert: DemoHighErrorCount
      expr: sum(ml_predictions_total{status="error"}) >= 3
      for: 10s
      labels:
        severity: demo
      annotations:
        summary: "[DEMO] Error threshold reached"
        description: "At least 3 prediction errors have occurred."

    # Fires with sustained traffic
    - alert: DemoHighRequestRate
      expr: sum(rate(ml_predictions_total[1m])) > 0.5
      for: 30s
      labels:
        severity: demo
      annotations:
        summary: "[DEMO] High request rate detected"
        description: "More than 0.5 requests per second for 30 seconds."
```

Alert anatomy:

- **expr**: PromQL expression that returns true when alert should fire
- **for**: How long the condition must be true before firing (prevents flapping)
- **labels**: Metadata for routing/filtering (e.g., severity)
- **annotations**: Human-readable descriptions

Part 5: Setting Up Grafana

Prometheus stores data; Grafana visualizes it.

Step 1: Create Directory Structure

```
mkdir -p grafana/provisioning/datasources
mkdir -p grafana/provisioning/dashboards
mkdir -p grafana/dashboards
```

Step 2: Configure Prometheus as Data Source

Create `grafana/provisioning/datasources/prometheus.yml`:

```
apiVersion: 1

datasources:
- name: Prometheus
  type: prometheus
  uid: prometheus           # Unique ID for referencing in dashboards
  access: proxy              # Grafana backend proxies requests
  url: http://prometheus:9090
  isDefault: true
  editable: false
  jsonData:
    timeInterval: "5s"
```

Step 3: Configure Dashboard Provisioning

Create `grafana/provisioning/dashboards/dashboards.yml`:

```
apiVersion: 1

providers:
- name: 'ML Monitoring Dashboards'
  orgId: 1
  folder: 'ML Monitoring'
  folderUid: 'ml-monitoring'
  type: file
  disableDeletion: false
  updateIntervalSeconds: 30
  options:
    path: /var/lib/grafana/dashboards
```

Step 4: Dashboard JSON

A pre-built dashboard JSON file (`grafana/dashboards/ml-api-dashboards.json`) is provided in the repository. It includes:

- **API Status:** UP/DOWN indicator
- **Total Predictions:** Counter
- **Error Rate:** Percentage over 2 minutes
- **P95 Latency:** 95th percentile response time
- **Memory Usage:** Current memory consumption
- **Request Rate Graph:** Requests per second over time
- **Latency Percentiles Graph:** p50, p95, p99 over time
- **Predictions by Result:** Pie chart breakdown
- **Predictions by Status:** Success vs error ratio
- **Active Alerts:** List of firing alerts

You can also create dashboards manually in the Grafana UI and export them as JSON.

Part 6: Docker Compose Configuration

Update your `docker-compose.yml` to include the monitoring stack:

```
version: '3.8'

services:
  # Your existing ML API
  app:
    build:
      context: .
      dockerfile: Dockerfile.mlapp
    container_name: ml-api
    ports:
      - "5000:5000"
    environment:
      - PORT=5000
      - LOG_DIR=/app/logs
      - MLFLOW_TRACKING_URI=http://mlflow:5001
    volumes:
      - ./logs:/app/logs
    networks:
      - ml-network
    depends_on:
      - mlflow
    restart: unless-stopped

  # Your existing MLflow
  mlflow:
    build:
      context: .
      dockerfile: Dockerfile.mlflow
    container_name: mlflow
    ports:
      - "5001:5001"
    volumes:
      - mlflow-data:/mlflow
    networks:
      - ml-network
    restart: unless-stopped

  # Prometheus - Metrics Collection
  prometheus:
    image: prom/prometheus:v2.47.0
    container_name: prometheus
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - ./prometheus/rules:/etc/prometheus/rules:ro
      - prometheus-data:/prometheus
```

```
command:
  - '--config.file=/etc/prometheus/prometheus.yml'
  - '--storage.tsdb.path=/prometheus'
  - '--web.console.libraries=/etc/prometheus/console_libraries'
  - '--web.console.templates=/etc/prometheus/consoles'
  - '--web.enable-lifecycle'
networks:
  - ml-network
depends_on:
  - app
restart: unless-stopped

# Grafana - Visualization
grafana:
  image: grafana/grafana:10.1.0
  container_name: grafana
  ports:
    - "3000:3000"
  environment:
    - GF_SECURITY_ADMIN_USER=admin
    - GF_SECURITY_ADMIN_PASSWORD=admin
    - GF_USERS_ALLOW_SIGN_UP=false
  volumes:
    - ./grafana/provisioning:/etc/grafana/provisioning:ro
    - ./grafana/dashboards:/var/lib/grafana/dashboards:ro
    - grafana-data:/var/lib/grafana
  networks:
    - ml-network
  depends_on:
    - prometheus
  restart: unless-stopped

networks:
  ml-network:
    driver: bridge

volumes:
  mlflow-data:
    driver: local
  prometheus-data:
    driver: local
  grafana-data:
    driver: local
```

Part 7: Running and Testing

Step 1: Build and Start All Services

```
docker-compose up --build -d
docker-compose ps # Verify all containers are running
```

All four containers should show as "Up" or "healthy".

Step 2: Access the Services

Service	URL	Credentials
ML API	http://localhost:5000	N/A
ML API Metrics	http://localhost:5000/metrics	N/A
Prometheus	http://localhost:9090	N/A
Grafana	http://localhost:3000	admin/admin
MLflow	http://localhost:5001	N/A

Step 3: Verify Prometheus Targets

1. Go to `http://localhost:9090/targets`
2. You should see two targets, both showing "UP":
 - `prometheus` (localhost:9090)
 - `ml-api` (app:5000)

If `ml-api` shows "DOWN", check that:

- The Flask app is running and healthy
- The `/metrics` endpoint is accessible
- The service name in `prometheus.yml` matches `docker-compose`

Step 4: Generate Test Traffic

Create a test data file (note: `SeniorCitizen` must be `"0"` or `"1"`, not `"No"/"Yes"`):

```
# Create test data file
echo
'{"tenure":12,"MonthlyCharges":59.95,"TotalCharges":720.50,"Contract":"One
year","PaymentMethod":"Electronic
check","OnlineSecurity":"No","TechSupport":"No","InternetService":"DSL","g
ender":"Female","SeniorCitizen":"0","Partner":"Yes","Dependents":"No","Pho
neService":"Yes","MultipleLines":"No","PaperlessBilling":"Yes","OnlineBack
up":"Yes","DeviceProtection":"No","StreamingTV":"No","StreamingMovies":"No
"}' > /tmp/test.json

# Health check
curl http://localhost:5000/health

# Make successful predictions
for i in {1..10}; do
  curl -s -X POST http://localhost:5000/v1/predict \
    -H "Content-Type: application/json" \
    -d @/tmp/test.json
```

```
echo ""  
done
```

Step 5: Verify Metrics are Being Collected

```
# Check metrics endpoint directly  
curl -s http://localhost:5000/metrics | grep ml_predictions_total  
  
# Check Prometheus has the data  
curl -s "http://localhost:9090/api/v1/query?query=ml_predictions_total" |  
jq '.data.result'
```

Step 6: Trigger Demo Alerts

To demonstrate alerting, trigger the `DemoHighErrorCount` alert:

```
# Send invalid requests to generate errors  
for i in {1..5}; do  
  curl -s -X POST http://localhost:5000/v1/predict \  
    -H "Content-Type: application/json" \  
    -d '{"invalid": "data"}'  
  echo ""  
done  
  
# Wait for alert to fire (10 second 'for' duration)  
sleep 15  
  
# Check alerts via API  
curl -s "http://localhost:9090/api/v1/alerts" | jq '.data.alerts'
```

You can also view alerts at <http://localhost:9090/alerts> in the Prometheus UI.

Step 7: Explore the Grafana Dashboard

1. Go to <http://localhost:3000>
2. Login with admin/admin (you'll be prompted to change the password)
3. Navigate to: Dashboards > ML Monitoring > ML API Monitoring Dashboard
4. You should see:
 - API Status showing "UP"
 - Total Predictions count
 - Error rate and latency metrics
 - Active alerts (if any are firing)

Part 8: PromQL Reference

PromQL (Prometheus Query Language) is how you query metrics.

Basic Queries

```
# Get current value of a metric
up{job="ml-api"}

# Get all prediction counts
ml_predictions_total

# Filter by label
ml_predictions_total{status="error"}
ml_predictions_total{model_version="v1", status="success"}
```

Rate and Aggregation

```
# Requests per second over last 5 minutes
rate(ml_predictions_total[5m])

# Total predictions across all labels
sum(ml_predictions_total)

# Group by a label
sum(ml_predictions_total) by (status)
sum(ml_predictions_total) by (model_version, prediction_result)
```

Percentiles from Histograms

```
# 95th percentile latency
histogram_quantile(0.95,
sum(rate(ml_prediction_duration_seconds_bucket[5m])) by (le))

# 50th percentile (median)
histogram_quantile(0.50,
sum(rate(ml_prediction_duration_seconds_bucket[5m])) by (le))
```

Useful Calculations

```
# Error rate as percentage
sum(rate(ml_predictions_total{status="error"}[5m]))
/ sum(rate(ml_predictions_total[5m])) * 100

# Average latency
rate(ml_prediction_duration_seconds_sum[5m])
/ rate(ml_prediction_duration_seconds_count[5m])
```

```
# Increase in last hour
increase(ml_predictions_total[1h])
```

Common Query Patterns

Pattern	Description
<code>metric_name</code>	Current value
<code>rate(counter[5m])</code>	Per-second rate over 5 minutes
<code>sum by(label) (metric)</code>	Sum grouped by label
<code>histogram_quantile(0.95, ...)</code>	95th percentile
<code>increase(counter[1h])</code>	Total increase over 1 hour
<code>avg_over_time(gauge[1h])</code>	Average gauge value over 1 hour

Troubleshooting

Prometheus Target Shows "DOWN"

1. Verify Flask app is running: `docker-compose ps`
2. Check `/metrics` endpoint: `curl http://localhost:5000/metrics`
3. Verify service name matches: `app:5000` in `prometheus.yml` vs. service name in `docker-compose`
4. Check Flask binds to `0.0.0.0` not `127.0.0.1`

No Data in Grafana

1. Verify Prometheus has data first: Run queries in Prometheus UI
2. Check data source configuration in Grafana (Settings > Data Sources)
3. Wait for scrape interval to pass (metrics aren't instant)
4. Check time range in Grafana (top right) - ensure it covers when data was generated

Alerts Not Firing

1. Check alert rules loaded: `http://localhost:9090/rules`
2. Verify metrics exist that the alert queries
3. Check the `for` duration - alert won't fire until condition is true for that long
4. Generate enough data to trigger the condition
5. **Common mistake:** Validation errors weren't being counted. Ensure ALL error paths increment the counter.

Grafana Password Issues

Reset to default:

```
docker-compose exec grafana grafana-cli admin reset-admin-password admin
```

Deliverables

1. **GitHub Repository** with:

- Instrumented Flask application (`src/app.py`) with Prometheus metrics
- Prometheus configuration (`prometheus/prometheus.yml`)
- Alert rules (`prometheus/rules/alerts.yml`)
- Grafana provisioning files (`grafana/provisioning/`)
- Dashboard JSON (`grafana/dashboards/`)
- Updated `docker-compose.yml`

2. **Screenshots** showing:

- Prometheus targets page (all targets "UP")
- A PromQL query result in Prometheus
- Grafana dashboard with live metrics
- At least one alert firing (e.g., `DemoHighErrorCount`)

3. **Brief Report** (1-2 pages) explaining:

- What metrics you chose to track and why
 - How monitoring helps detect issues in ML systems
 - Any challenges you encountered and how you resolved them
-

Evaluation Criteria

Criteria	Points
Prometheus metrics correctly implemented in Flask	25
Prometheus configuration and scraping working	20
Grafana dashboard setup and visualization	20
Successfully triggered and documented alerts	15
Report quality and understanding demonstrated	15
Code quality and best practices	5
Total	100

Additional Resources

- [Prometheus Documentation](#)
- [Grafana Documentation](#)
- [prometheus-flask-exporter GitHub](#)
- [PromQL Cheat Sheet](#)
- [Prometheus Best Practices](#)